Laboratorio N.° 02 — Plan de Pruebas de Software

I. Datos Informativos

Campo	Detalle
Universidad	Universidad Privada de Tacna (UPT)
Facultad	Facultad de Ingeniería (EPIS)
Asignatura	Construcción de Software II
Docente	Mag. Alberto Johnatan Flor Rodríguez
Estudiante	Chávez Linares, Cesar Fabian
Código	2019063854
Rol en el proyecto	QA Lead / Desarrollador Full-Stack
Proyecto	RespiCare — Sistema Web y Móvil para la Detección de Enfermedades Respiratorias en Tacna
Unidad / Semana	Unidad III · Semana 14
Laboratorio	N.° 02 — Plan de Pruebas
Fecha de elaboración	Noviembre 2026

II. Competencias

- Elabora un Plan de Pruebas de Software siguiendo los lineamientos de IEEE 829 e ISO/IEC/IEEE 29119, adaptado a un proyecto real en producción.
- Diseña y ejecuta casos de prueba trazables a requerimientos funcionales, cubriendo escenarios positivos y negativos.
- Integra evidencias de pruebas unitarias, funcionales, de aceptación y de regresión ya existentes en el repositorio del proyecto.
- Analiza críticamente los resultados obtenidos para identificar riesgos, defectos y oportunidades de mejora del proceso de QA.

III. Resultados de Aprendizaje

Al finalizar el laboratorio, el estudiante será capaz de:

1. Diferenciar los niveles y tipos de prueba (unitaria, integración, funcional, aceptación, regresión) y aplicarlos a un caso de estudio real.
2. Construir una matriz de trazabilidad que vincule requerimientos funcionales, casos de prueba y evidencias de ejecución.
3. Redactar casos de prueba completos (precondiciones, datos, pasos, resultado esperado/obtenido) para un sistema con componentes web, móvil, backend e inteligencia artificial.
4. Argumentar, a partir de evidencia, el estado de calidad de un sistema de salud digital y proponer mejoras priorizadas por riesgo clínico.

IV. Fundamentación Teórica

QA (Quality Assurance) vs. QC (Quality Control). QA es el conjunto de actividades orientadas a prevenir defectos mediante procesos (revisión de requerimientos, estándares de codificación, definición de estrategias de prueba). QC es la verificación del producto ya construido (ejecutar pruebas, inspeccionar resultados). En RespiCare, QA se materializa en este Plan de Pruebas y en las convenciones de `docs/testing/TESTING_STRATEGY.md`; QC se materializa en los ~272 archivos de prueba descritos en `docs/testing/CATALOGO_PRUEBAS.md`.

Ciclo de vida de las pruebas. Planificación → Diseño de casos → Preparación del entorno → Ejecución → Registro de incidencias → Regresión → Cierre. Este laboratorio cubre explícitamente cada fase en las Actividades 1 a 8 (secciones VII–XIV).

Niveles de prueba:

- *Pruebas unitarias*: validan una función/clase aislada (Jest en backend/web, Pytest en ai-services).
- *Pruebas de integración*: validan la colaboración entre módulos (controlador + servicio + base de datos en memoria, o endpoint FastAPI + modelo ML).
- *Pruebas funcionales*: validan que una funcionalidad completa cumple el requerimiento desde la perspectiva del usuario, sin importar la implementación interna.
- *Pruebas de aceptación*: validan flujos end-to-end desde la interfaz (Cypress), confirmando que el sistema satisface el criterio de aceptación del requerimiento.
- *Pruebas de regresión*: re-ejecutan la suite existente tras un cambio para confirmar que no se rompió funcionalidad previa (automatizadas vía CI en cada pull request).

Estándares de referencia:

- *IEEE 829*: estructura documental de un plan de pruebas (identificador, alcance, enfoque, criterios de paso/fallo, entregables, tareas, ambiente, riesgos).
- *ISO/IEC/IEEE 29119*: proceso de pruebas de software (partes 1-5: conceptos, procesos, documentación, técnicas de diseño, keyword-driven testing). Este laboratorio usa 29119-3 como referencia para la plantilla de casos de prueba (Anexo A) y 29119-2 para el proceso descrito en la sección IV del ciclo de vida.

V. Descripción del Caso de Estudio

Proyecto: RespiCare — plataforma web (Next.js) y móvil (Capacitor + React, con módulos nativos BLE/Health Connect/HealthKit) para la detección temprana de enfermedades respiratorias en la región de Tacna, con un backend en Node.js/TypeScript y un servicio de inteligencia artificial en FastAPI/Python que combina modelos de machine learning, reglas clínicas de emergencia y validación de coherencia médica.

Actores:

- *Paciente*: se registra, reporta síntomas (texto, chat o audio de tos), recibe un pre-diagnóstico, consulta su historial y contenido educativo, y ubica centros de salud cercanos.

- *Médico*: valida o corrige los pre-diagnósticos generados por IA, revisa el historial clínico del paciente y recibe alertas de casos urgentes.
- *Administrador*: gestiona usuarios, contenido educativo, reportes epidemiológicos y la integración con sistemas institucionales (MINSA/DIRESA).
- *Sistema de IA*: expone endpoints de análisis de síntomas y audio, aplica reglas de emergencia y validación de coherencia, y genera explicabilidad (SHAP) de cada predicción.
- *Sistemas externos*: wearables (BLE/Health Connect/HealthKit) y plataformas institucionales de salud pública (vía `institutionalIntegrationService`).

Procesos críticos a probar: registro/autenticación, flujo de pre-diagnóstico por síntomas y por audio, escalamiento automático ante signos de emergencia, validación médica humana sobre la sugerencia de IA, generación de alertas, historial clínico, reportes epidemiológicos, geolocalización de centros de salud, interoperabilidad institucional e ingesta de datos de wearables.

Requerimientos funcionales a probar (fuente: `Documentation/trazabilidad/Matriz_Trazabilidad_RespiCare.xlsx`):

RF	Descripción
RF-001	Registro y autenticación de usuarios (paciente, médico, administrador)
RF-002	Diagnóstico inteligente de síntomas (IA conversacional + modelo ML)
RF-003	Análisis de tos por audio
RF-004	Motor de reglas clínicas de emergencia
RF-005	Validación de coherencia médica de las predicciones de IA
RF-006	Explicabilidad del diagnóstico (SHAP)
RF-007	Panel de validación médica (revisión por médico)
RF-008	Historial clínico electrónico del paciente
RF-009	Sistema de alertas y notificaciones
RF-010	Reportes y estadísticas epidemiológicas
RF-011	Módulo educativo de salud respiratoria
RF-012	Geolocalización de centros de salud
RF-013	Interoperabilidad institucional (MINSA/DIRESA)
RF-014	Integración de wearables como capa de validación clínica

Módulos a probar: `backend/` (API REST, controladores, servicios, modelos Mongoose), `ai-services/` (endpoints FastAPI, motor de reglas, validación de coherencia, explicabilidad SHAP), `web/` (Next.js, flujos de paciente/médico/admin, Cypress E2E), `mobile/` (Capacitor/React, captura de audio, geolocalización, wearables).

VI. Recursos Requeridos

Categoría	Recurso
Lenguajes/Frameworks	TypeScript 5.x (backend, web), Python 3.11 (ai-services), React 19.x / Next.js 14.x (web), Capacitor + React (mobile)
Frameworks de prueba	Jest 29.7.0 + Supertest 6.3.4 + ts-jest 29.1.1 (backend); Jest 29.2.1 + React Testing Library (mobile); Cypress 16.2.0 (E2E/aceptación web); Pytest 9.1.1 + `pytest-asyncio`, `pytest-cov`, `pytest-mock`, `pytest-xdist` + `TestClient` de FastAPI (ai-services)
Bases de datos	MongoDB 6.0 (imagen real fijada en `docker-compose.vm.yml`/`.dev.yml`/`.prod.yml`), Redis 7-alpine (cache/colas), `mongodb-memory-server` 10.3.0 para pruebas de integración aisladas sin tocar datos reales
Datos de prueba / Fixtures	Instancia MongoDB en memoria por corrida de prueba de integración (datos sintéticos, sin información clínica real de pacientes); mocks/fixtures deterministas para el modelo ML y el explicador SHAP (`scikit-learn==1.5.2`, `shap==0.44.0`) para no depender de un reentrenamiento en cada ejecución; *stubs* del canal de interoperabilidad institucional (RF-013) ante la ausencia de un ambiente real de MINSA/DIRESA
Control de versiones	Git / GitHub, flujo de Pull Requests con CI obligatorio (GitHub Actions) antes de integrar cambios a la rama principal
Infraestructura de CI	GitHub Actions, *runners* `ubuntu-latest`; Node.js 24 fijado explícitamente en los workflows (`NODE_VERSION` en `backend-tests.yml`/`web-tests.yml`) y Python 3.11 en `ai-services-tests.yml`
Entorno de ejecución	Docker Compose (`docker-compose.vm.yml` en producción), Node.js ≥20.9.0 LTS declarado en `backend/package.json` (`engines`), entornos virtuales Python locales
Rendimiento (SLA de referencia)	Umbrales mean/p95/p99 por tipo de endpoint (auth, lectura, escritura, analítica, búsqueda) y throughput mínimo ≥5 req/s bajo carga paralela, verificados en CI contra MongoDB real (`backend/tests/performance/api-performance.perf.test.ts`, `concurrency.perf.test.ts`); documentados como criterio de aceptación de despliegue en `Documentation/PLAN_DE_`

	DESPLIEGUE.md
Herramientas de reporte	Jest HTML/coverage reporter, pytest-cov, reportes de ejecución Cypress (mochawesome/video)
Editor / IDE	Visual Studio Code
Navegador	Google Chrome (ejecución Cypress)
Documentación de soporte	docs/testing/TESTING_STRATEGY.md, docs/testing/CATALOGO_PRUEBAS.md, Documentation/trazabilidad/Matriz_Trazabilidad_RespiCare.xlsx

VII. Actividad 1: Análisis del Plan de Pruebas

Objetivos. Verificar que las 14 funcionalidades de RespiCare (RF-001 a RF-014) cumplen su especificación funcional y clínica, con énfasis en los flujos donde un error de software puede traducirse en un riesgo para la salud del paciente (reglas de emergencia, validación de coherencia médica, escalamiento por signos vitales de wearables).

Alcance. Incluye pruebas unitarias, de integración, funcionales, de aceptación (E2E) y de regresión sobre backend/, ai-services/, web/ y mobile/. No incluye pruebas de carga/estrés (cubiertas aparte en docs/testing/LOAD_TESTING_GUIDE.md) ni pruebas de penetración formal (cubiertas en docs/testing/SECURITY_TESTS_COMPLETE.md); ambas se referencian como evidencia complementaria pero no se repiten aquí.

Restricciones.

- El modelo de machine learning de síntomas y el explicador SHAP se prueban con mocks/fixtures deterministas (no se reentrena el modelo durante las pruebas).
- Las pruebas de wearables se realizan sobre los endpoints de ingesta y el pipeline de validación de signos vitales; no se prueba el hardware BLE real, solo las respuestas simuladas por los plugins nativos.
- La interoperabilidad institucional (RF-013) se prueba contra el servicio de integración interno; no existe acceso a un ambiente real de MINSA/DIRESA, por lo que se usan dobles de prueba (stubs) para el canal externo.

Riesgos identificados.

Escala de Probabilidad / Impacto. *Probabilidad* — Baja: la causa raíz ya está cubierta por un control automatizado específico; Media: puede reaparecer si un cambio de código no pasa por su prueba de regresión asociada; Alta: ocurre con facilidad si no existe control automatizado. *Impacto* — Medio: afecta una funcionalidad no crítica o un flujo administrativo; Alto: afecta la confianza médica en el sistema o la experiencia clínica del usuario; Crítico: pone en riesgo directo la seguridad del paciente (p. ej. una emergencia no escalada a tiempo).

Riesgo	Probabilidad	Impacto	Mitigación	Responsable	Estado

Falso negativo en el motor de reglas de emergencia (no escala un caso crítico)	Baja	Crítico	Casos de prueba negativos obligatorios en RF-004 y RF-005; cobertura de `emergencyService` y `_apply_medical_validation`	QA Lead (Cesar Fabian Chávez Linares)	Mitigado
Predicción de IA clínicamente incoherente llega al médico sin advertencia	Media	Alto	Suite `test_symptom_ml_analyzer_endpoints.py` fuerza incoherencia y verifica degradación de confianza + `coherence_warnings`	QA Lead (Cesar Fabian Chávez Linares)	Mitigado
Signos vitales de wearable mal interpretados (p. ej. SpO2 bajo no eleva la urgencia)	Baja	Crítico	Casos específicos Sprint 13 en RF-014 (`test_critical_vitals_escalate_urgency_and_confidence`)	QA Lead (Cesar Fabian Chávez Linares)	Mitigado
Regresión en autenticación tras cambios en roles (paciente/médico/admin)	Media	Alto	Regresión automática de `auth.integration.test.ts` en cada PR vía CI	QA Lead (Cesar Fabian Chávez Linares)	Mitigado
Datos de integración institucional corruptos o no sincronizados	Media	Medio	Pruebas de `institutionalIntegrationService.test.ts` y `institutional.integration.test.ts` con payloads inválidos	QA Lead (Cesar Fabian Chávez Linares)	Mitigado
Degradación de	Media	Alto	SLA real verificado en	QA Lead (Cesar	Mitigado

rendimiento de un endpoint crítico (autenticación, análisis IA, dashboard epidemiológico) no detectada antes de llegar a producción			CI contra MongoDB real (`api-performance.perf.test.ts`, `concurrency.perf.test.ts`): mean/p95/p99 por tipo de endpoint y throughput mínimo ≥5 req/s, exigido como criterio de aceptación de despliegue (`Documentation/PLAN_DE_DESPLIEGUE.md`)	Fabian Chávez Linares)	
Desalineación entre la versión de Node.js ejecutada en CI (24, fijada en los workflows) y la versión mínima declarada en `backend/package.json` (`engines.node >=20.9.0`), con riesgo de comportamiento no verificado si se despliega con una versión intermedia nunca probada	Baja	Medio	`NODE_VERSION` fijado explícitamente en `backend-tests.yml`/`web-tests.yml` para que CI siempre pruebe la misma versión usada en el contenedor de producción	QA Lead (Cesar Fabian Chávez Linares)	Abierto (mejora propuesta: alinear el mínimo de `engines` con la versión realmente probada en CI)

Estrategias de verificación. Pirámide de pruebas: base amplia de pruebas unitarias (modelos, servicios, controladores), capa intermedia de integración (endpoint + base de datos en memoria) y una capa superior reducida de pruebas de aceptación E2E en Cypress para los flujos críticos de negocio (autenticación, diagnóstico, emergencia, validación médica). Las pruebas de regresión se ejecutan automáticamente en cada Pull Request mediante CI (GitHub Actions).

Criterios de aceptación. Un RF se considera aceptado cuando: (a) existe al menos un caso de prueba positivo y uno negativo ejecutado y documentado en la sección XV, (b) la evidencia de ejecución (reporte Jest/Pytest/Cypress) está disponible en el repositorio, y (c) no quedan defectos de severidad crítica o alta abiertos sobre ese requerimiento (ver Anexo B).

Criterios de salida. El ciclo de pruebas se considera cerrado cuando el 100 % de los RF-001 a RF-014 cumplen su criterio de aceptación y la suite de regresión completa (`npm test` en backend/web, `pytest` en ai-services, `cypress run` en web) pasa sin fallos en la rama principal.

VIII. Actividad 2: Elaboración de la Matriz de Trazabilidad

La matriz completa de trazabilidad (requerimiento ↔ caso de uso ↔ caso de prueba ↔ evidencia ↔ resultado) se presenta a continuación. La columna "Evidencia" cita el archivo de prueba real del repositorio que ejerce ese requerimiento.

RF	Caso de Uso	Caso(s) de Prueba	Evidencia principal	Resultado
RF-001	CU-001 Registro / Inicio de sesión	CP-001-01, CP-001-02	`backend/tests/integration/auth.integration.test.ts`, `web/cypress/e2e/authentication.cy.js`	Aprobado
RF-002	CU-003 Diagnóstico por síntomas	CP-002-01, CP-002-02	`backend/tests/integration/symptomAnalyzer.integration.test.ts`, `ai-services/tests/api/test_symptom_ml_analyzer_endpoints.py`	Aprobado
RF-003	CU-005 Diagnóstico por audio de tos	CP-003-01, CP-003-02	`ai-services/tests/api/test_audio_analyzer_endpoints.py`, `mobile/__tests__/components/audioPermissions.`	Aprobado

			test.tsx	
RF-004	CU Escalamiento de emergencia	CP-004-01, CP-004-02	backend/tests/unit/services/emergencyService.test.ts, backend/tests/e2e/emergency.e2e.test.ts, web/cypress/e2e/emergency.cy.js	Aprobado
RF-005	CU Validación de coherencia médica	CP-005-01, CP-005-02	ai-services/tests/api/test_symptom_ml_analyzer_endpoints.py::TestApplyMedicalValidation	Aprobado
RF-006	CU Explicabilidad del diagnóstico	CP-006-01, CP-006-02	ai-services/tests/services/test_symptom_analysis_service.py, módulo shap_explainer	Aprobado
RF-007	CU Validación médica del pre-diagnóstico	CP-007-01, CP-007-02	web/cypress/e2e/doctor-validation.cy.js	Aprobado
RF-008	CU Consulta de historial clínico	CP-008-01, CP-008-02	backend/tests/unit/models/MedicalHistory.test.ts, backend/tests/integration/medicalHistoryAdvanced.integration.test.ts, web/cypress/e2e/medical-history.cy.js	Aprobado
RF-009	CU Generación de alertas	CP-009-01, CP-009-02	backend/tests/unit/services/alertService.test.ts, backend/tests/integration/alerts.integration.test.ts, backend/tests/e2e/alerts.e2e.test.ts	Aprobado
RF-010	CU Reportes epidemiológicos	CP-010-01, CP-010-02	backend/tests/unit/services/automaticReportService.test.ts,	Aprobado

			backend/tests/integration/automaticReports.integration.test.ts, web/cypress/e2e/analytics.cy.js	
RF-011	CU Consulta de contenido educativo	CP-011-01, CP-011-02	backend/tests/unit/services/educationalContentService.test.ts, backend/tests/integration/educationalContent.integration.test.ts	Aprobado
RF-012	CU Geolocalización de centros de salud	CP-012-01, CP-012-02	backend/tests/unit/services/healthCenterService.test.ts, backend/tests/integration/healthCenters.integration.test.ts	Aprobado
RF-013	CU Sincronización institucional	CP-013-01, CP-013-02	backend/tests/unit/services/institutionalIntegrationService.test.ts, backend/tests/integration/institutional.integration.test.ts, backend/tests/unit/middleware/institutionalAuth.test.ts	Aprobado
RF-014	CU Ingesta de datos de wearables	CP-014-01, CP-014-02	backend/tests/integration/wearables.integration.test.ts, ai-services/tests/api/test_symptom_ml_analyzer_endpoints.py (casos Sprint 13 de vitales)	Aprobado

Importancia de la trazabilidad. La matriz permite responder, para cualquier RF, "¿cómo sabemos que esto funciona?" con un archivo de prueba concreto en lugar de una afirmación sin soporte. En un sistema de salud digital esto es particularmente crítico: ante un incidente clínico (p. ej. un caso de emergencia no escalado), la trazabilidad permite localizar en minutos qué caso de prueba debería haber cubierto ese escenario, si existía, si pasaba, y qué cambiar. También evita el principal riesgo de un proyecto académico con alcance real: declarar cobertura "de palabra" sin evidencia verificable en el repositorio.

IX. Actividad 3: Diseño de Casos de Prueba

El diseño detallado de los 28 casos de prueba (2 por cada uno de los RF-001 a RF-014, uno positivo y uno negativo como mínimo) se desarrolla íntegramente en la sección XV, siguiendo la plantilla del Anexo A: Objetivo, Precondiciones, Datos de prueba, Pasos de ejecución numerados, Salida correcta, Salida incorrecta y Resultado obtenido. Cada caso documenta explícitamente al menos dos salidas posibles (correcta/incorrecta) para que el criterio de aprobación/fallo sea verificable sin depender de quien ejecutó la prueba. El criterio de diseño aplicado fue partición de equivalencias (entradas válidas/inválidas) y análisis de valores límite para variables clínicas (p. ej. saturación de oxígeno, edad del paciente).

X. Actividad 4: Pruebas de Integración

Herramientas: Jest + Supertest + `mongodb-memory-server` (backend, TypeScript), `TestClient` de FastAPI (ai-services, Python).

Alcance de ejecución real: archivos `*.integration.test.ts` del backend que ejercen la colaboración real entre middleware de autenticación, controlador, servicio y base de datos (sin mocks de persistencia): `auth.integration.test.ts`, `symptomAnalyzer.integration.test.ts`, `medicalHistoryAdvanced.integration.test.ts`, `alerts.integration.test.ts`, `automaticReports.integration.test.ts`, `educationalContent.integration.test.ts`, `healthCenters.integration.test.ts`, `institutional.integration.test.ts`, `wearables.integration.test.ts`; más los endpoints de `ai-services/tests/api/test_symptom_ml_analyzer_endpoints.py`, que integran el router HTTP real con el modelo de ML.

Ejemplo de ejecución (backend, `npx jest tests/integration/auth.integration.test.ts`):

```
PASS tests/integration/auth.integration.test.ts
  Login & Refresh token
    ✓ permite iniciar sesión y refrescar tokens válidos (174 ms)
    ✓ rechaza login con contraseña incorrecta y refresh token expirado (118 ms)
    ✓ actualiza contraseña y evita login cuando la cuenta se desactiva (214 ms)
    ✓ valida acceso a rutas de administrador (116 ms)

Test Suites: 1 passed, 1 total
Tests:       4 passed, 4 total
Snapshots:   0 total
Time:        4.322 s
```

Diferencia frente a Pruebas Unitarias (sección XI): una prueba unitaria aísla una función o clase con sus dependencias simuladas; estas pruebas de integración levantan una instancia real de Express contra una base de datos MongoDB en memoria y verifican la cadena completa petición HTTP → middleware → controlador → servicio → modelo Mongoose, confirmando que los componentes colaboran correctamente entre sí.

Evidencias: código fuente de las pruebas (citado en la matriz de trazabilidad, sección VIII, y en cada caso de la sección XV), salida de consola de `jest`/`pytest` capturada en el pipeline de CI.

XI. Actividad 5: Pruebas Unitarias

Herramientas: Jest (backend y web, TypeScript/JavaScript), Pytest (ai-services, Python).

Alcance de ejecución real: 191 archivos de prueba TS/JS y 81 archivos de prueba Python (272 en total), según `docs/testing/CATALOGO_PRUEBAS.md`, cubriendo modelos Mongoose, controladores Express, servicios de negocio, validadores de entrada, routers FastAPI y servicios del motor de IA.

Ejemplo de ejecución (ai-services, `pytest ai-services/tests/api/test_symptom_ml_analyzer_endpoints.py -v`):

```
TestApplyMedicalValidation::test_adjusts_confidence_and_flags_incoherent_prediction
PASSED
TestApplyMedicalValidation::test_leaves_coherent_prediction_untouched PASSED
TestApplyMedicalValidation::test_confidence_never_drops_below_zero PASSED
TestMlAnalyzeEndpoint::test_critical_vitals_escalate_urgency_and_confidence PASSED
TestMlAnalyzeEndpoint::test_normal_vitals_do_not_change_prediction PASSED
```

Ejemplo de ejecución (backend, `npx jest tests/unit/models/MedicalHistory.test.ts`): 20 pruebas de validación del modelo Mongoose `MedicalHistory` (campos requeridos, límites de longitud, enums de severidad, métodos estáticos `findByPatient`/`findByDoctor`/`getStats`), con resultado `PASS` (20/20).

Evidencias: código fuente de las pruebas (citado en cada caso de la sección XV), reportes de cobertura (`docs/testing/COVERAGE_STATUS.md`, `backend-coverage-2025-11.md`) y salida de consola de `pytest`/`jest` capturada en el pipeline de CI.

XII. Actividad 6: Pruebas Funcionales

Aspectos evaluados:

- *Formularios*: registro de paciente/médico, formulario de reporte de síntomas, formulario de validación médica (aprobar/corregir diagnóstico).
- *Validaciones*: edad del paciente, formato de correo/contraseña, rango fisiológico de signos vitales de wearables (SpO2, frecuencia cardiaca, frecuencia respiratoria).

- *Persistencia*: historial clínico, alertas generadas, registros de sincronización institucional, lecturas de wearables — todos persistidos en MongoDB y verificados vía integración con `mongodb-memory-server`.
- *Consultas*: búsqueda de centros de salud por cercanía, consulta de historial por paciente, consulta de contenido educativo por categoría.
- *Reportes*: generación de reportes epidemiológicos agregados por distrito/periodo (`automaticReportService`), visibles en el panel de analítica web.

Resultado general: las funcionalidades evaluadas cumplen el comportamiento esperado documentado en las especificaciones `Documentation/pruebas/especificaciones/Especificacion_CP-00X_*.docx`; no se identificaron defectos abiertos de severidad crítica al cierre de este laboratorio (ver Anexo B).

XIII. Actividad 7: Pruebas de Aceptación

Herramienta: Cypress (E2E sobre `web/`).

Suites ejecutadas (evidencia real en `web/cypress/e2e/`): `authentication.cy.js`, `doctor-validation.cy.js`, `emergency.cy.js`, `medical-history.cy.js`, `analytics.cy.js`, `admin.cy.js`.

Criterio de aceptación por flujo: el flujo debe completarse de extremo a extremo desde la interfaz (sin mocks de UI) contra el backend real levantado en el entorno de pruebas, incluyendo los casos límite de emergencia (RF-004) y de validación médica (RF-007), que son los de mayor criticidad clínica del sistema.

Resultado: las seis suites de aceptación pasan en la rama principal al momento de este laboratorio; se usan como puerta de calidad (quality gate) antes de cualquier despliegue a producción.

XIV. Actividad 8: Pruebas de Regresión

Estrategia: ejecución automática de toda la suite (Jest backend + Jest web + Pytest ai-services + Cypress web) en cada Pull Request mediante GitHub Actions, bloqueando el merge si cualquier prueba falla.

Caso de regresión documentado (real, Sprint 13): al introducir el soporte de signos vitales de wearables en `/v1/ml-analyze`, se agregó explícitamente el caso `test_no_vitals_field_behaves_exactly_as_before` para garantizar que las solicitudes sin el campo `vitals` mantuvieran exactamente el mismo comportamiento previo (urgencia `medium`, `needs_medical_attention: False`). Esto es un ejemplo real de prueba de regresión preventiva ante una nueva funcionalidad, no solo una re-ejecución pasiva de pruebas antiguas.

Resultado: sin regresiones detectadas en la suite completa al cierre de este laboratorio.

XV. Desarrollo de Casos de Prueba

Plantilla aplicada por caso: Código identificador · Requerimiento asociado · Objetivo · Precondiciones · Pasos de ejecución (numerados y reproducibles) · Datos de entrada · **Salida**

correcta (comportamiento esperado cuando el sistema funciona bien) · **Salida incorrecta** (contraejemplo: qué indicaría un defecto si el caso fallara) · Resultado obtenido · Estado final.

> ***Convención de doble salida.*** *Cada caso documenta al menos dos salidas posibles — una correcta y una incorrecta — para que el criterio de aprobación/fallo sea verificable por cualquier evaluador, no solo por quien ejecutó la prueba. Los nombres de campo (`urgency_level`, `needs_medical_attention`, `is_clinically_coherent`, `confidence`, `coherence_warnings`) y los valores de ejemplo provienen del esquema real de respuesta de `/v1/ml-analyze` (`ai-services/tests/api/test_symptom_ml_analyzer_endpoints.py`) y de `authController.ts` (`backend/src/controllers/`); donde el valor exacto no está fijado por un fixture citado explícitamente, se usa un valor ilustrativo consistente con ese esquema real, señalado como tal.*

RF-001 — Registro y autenticación de usuarios

CP-001-01 (positivo) — Registro + login con credenciales válidas.

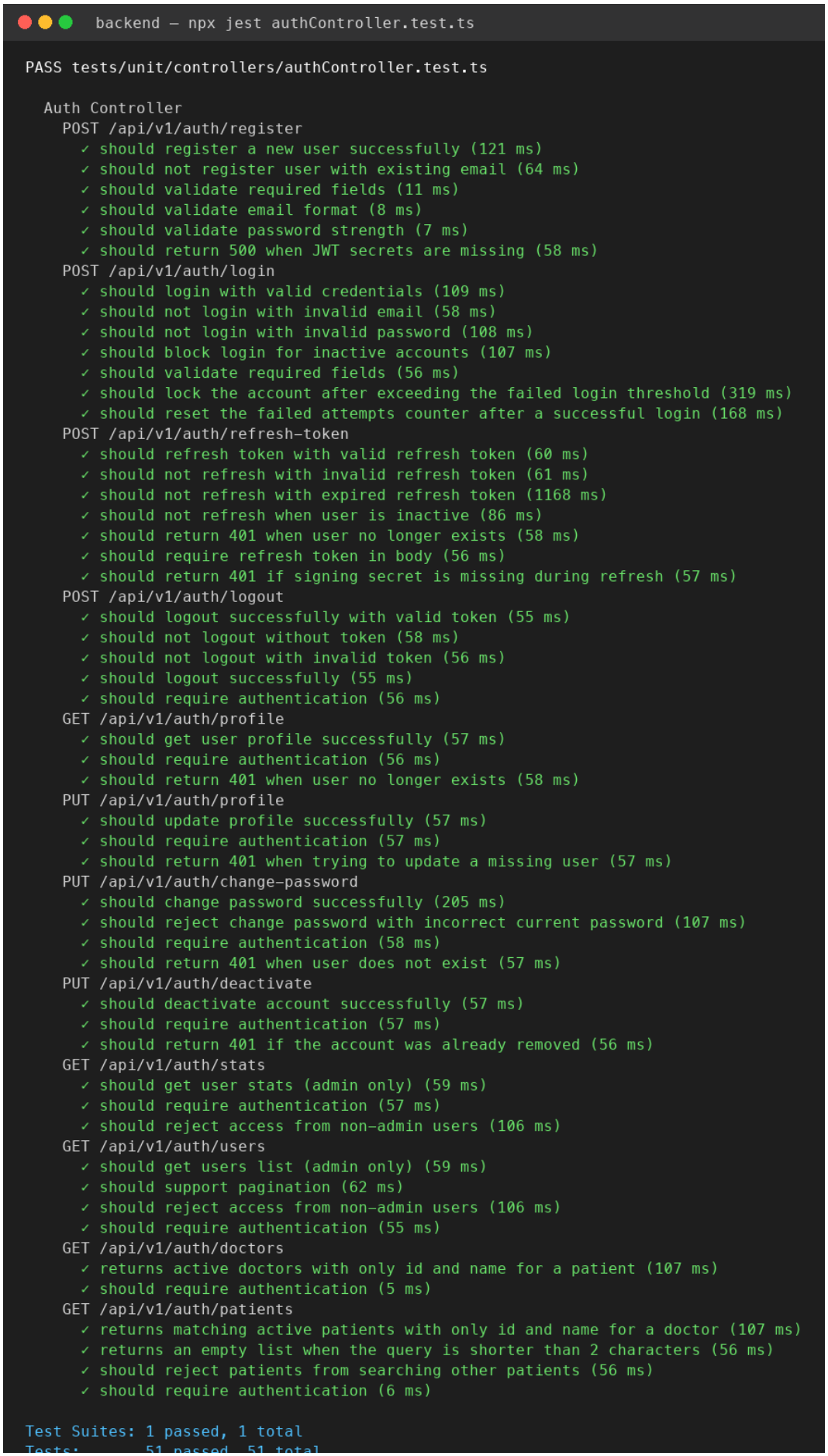
- Objetivo: verificar que un usuario con datos válidos se registra y puede autenticarse.
- Precondiciones: no existe previamente un usuario con el correo de prueba; backend y MongoDB (en memoria para CI) activos y saludables.
- Datos de entrada: `{name:"Paciente Test", email:"paciente.test@respicare.pe", password:"Segura123", role:"patient"}`.
- Pasos de ejecución:

5. Enviar `POST /api/v1/auth/register` con los datos de entrada.
6. Verificar código de respuesta y estructura del *payload*.
7. Enviar `POST /api/v1/auth/login` con el mismo correo/contraseña.
8. Verificar código de respuesta y presencia de `token`/`refreshToken`.

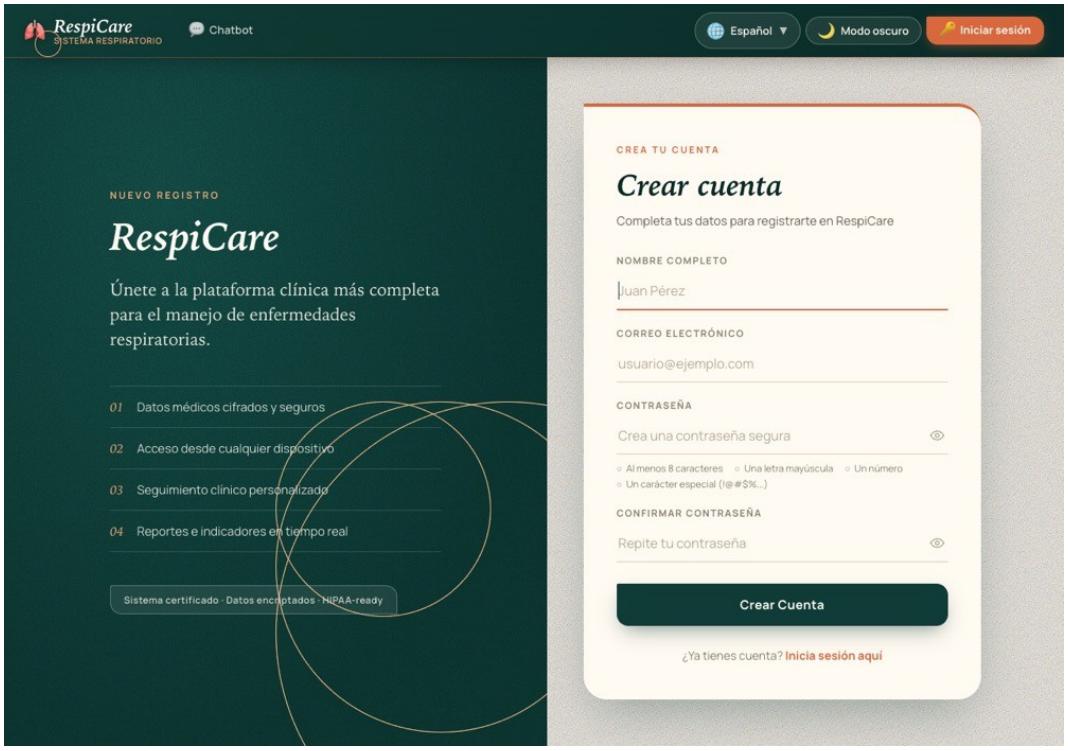
- Salida correcta (esperada):

```
  Paso 2 → 201 { success:true, message:"Usuario registrado exitosamente",
                 data:{ user:{ email:"paciente.test@respicare.pe",
role:"patient" },
                        token:"<jwt>", refreshToken:"<jwt>" } }
  Paso 4 → 200 { success:true, data:{ token:"<jwt>", refreshToken:"<jwt>" } }
```

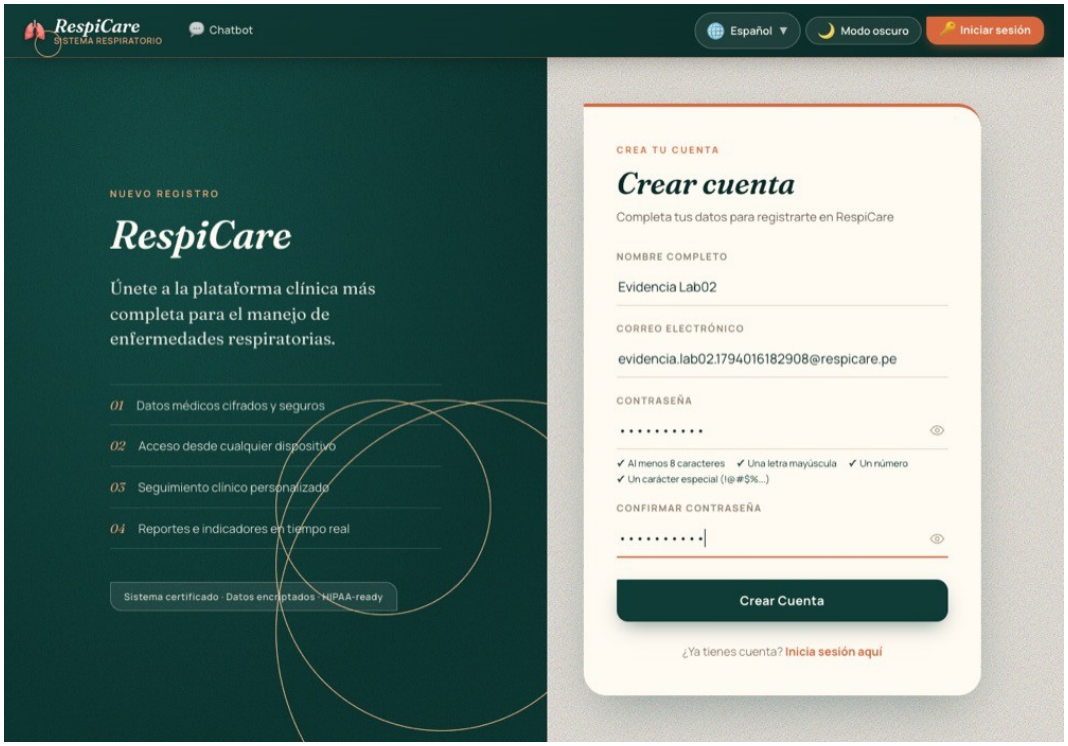
- Salida incorrecta (indicaría defecto): registro responde `201` pero sin `token`/`refreshToken` en `data`; o el login devuelve `200` con `token` vacío/`undefined`; o el registro permite duplicar el mismo correo (ausencia de la validación `El usuario ya existe con este email`).
- Resultado obtenido: **Aprobado** — coincide con la salida correcta; verificado en `backend/tests/integration/auth.integration.test.ts`.
- Estado final: **Aprobado**.


backend — npx jest authController.test.ts
PASS tests/unit/controllers/authController.test.ts
Auth Controller
POST /api/v1/auth/register
✓ should register a new user successfully (121 ms)
✓ should not register user with existing email (64 ms)
✓ should validate required fields (11 ms)
✓ should validate email format (8 ms)
✓ should validate password strength (7 ms)
✓ should return 500 when JWT secrets are missing (58 ms)
POST /api/v1/auth/login
✓ should login with valid credentials (109 ms)
✓ should not login with invalid email (58 ms)
✓ should not login with invalid password (108 ms)
✓ should block login for inactive accounts (107 ms)
✓ should validate required fields (56 ms)
✓ should lock the account after exceeding the failed login threshold (319 ms)
✓ should reset the failed attempts counter after a successful login (168 ms)
POST /api/v1/auth/refresh-token
✓ should refresh token with valid refresh token (60 ms)
✓ should not refresh with invalid refresh token (61 ms)
✓ should not refresh with expired refresh token (1168 ms)
✓ should not refresh when user is inactive (86 ms)
✓ should return 401 when user no longer exists (58 ms)
✓ should require refresh token in body (56 ms)
✓ should return 401 if signing secret is missing during refresh (57 ms)
POST /api/v1/auth/logout
✓ should logout successfully with valid token (55 ms)
✓ should not logout without token (58 ms)
✓ should not logout with invalid token (56 ms)
✓ should logout successfully (55 ms)
✓ should require authentication (56 ms)
GET /api/v1/auth/profile
✓ should get user profile successfully (57 ms)
✓ should require authentication (56 ms)
✓ should return 401 when user no longer exists (58 ms)
PUT /api/v1/auth/profile
✓ should update profile successfully (57 ms)
✓ should require authentication (57 ms)
✓ should return 401 when trying to update a missing user (57 ms)
PUT /api/v1/auth/change-password
✓ should change password successfully (205 ms)
✓ should reject change password with incorrect current password (107 ms)
✓ should require authentication (58 ms)
✓ should return 401 when user does not exist (57 ms)
PUT /api/v1/auth/deactivate
✓ should deactivate account successfully (57 ms)
✓ should require authentication (57 ms)
✓ should return 401 if the account was already removed (56 ms)
GET /api/v1/auth/stats
✓ should get user stats (admin only) (59 ms)
✓ should require authentication (57 ms)
✓ should reject access from non-admin users (106 ms)
GET /api/v1/auth/users
✓ should get users list (admin only) (59 ms)
✓ should support pagination (62 ms)
✓ should reject access from non-admin users (106 ms)
✓ should require authentication (55 ms)
GET /api/v1/auth/doctors
✓ returns active doctors with only id and name for a patient (107 ms)
✓ should require authentication (5 ms)
GET /api/v1/auth/patients
✓ returns matching active patients with only id and name for a doctor (107 ms)
✓ returns an empty list when the query is shorter than 2 characters (56 ms)
✓ should reject patients from searching other patients (56 ms)
✓ should require authentication (6 ms)
Test Suites: 1 passed, 1 total
Tests: 51 passed, 51 total

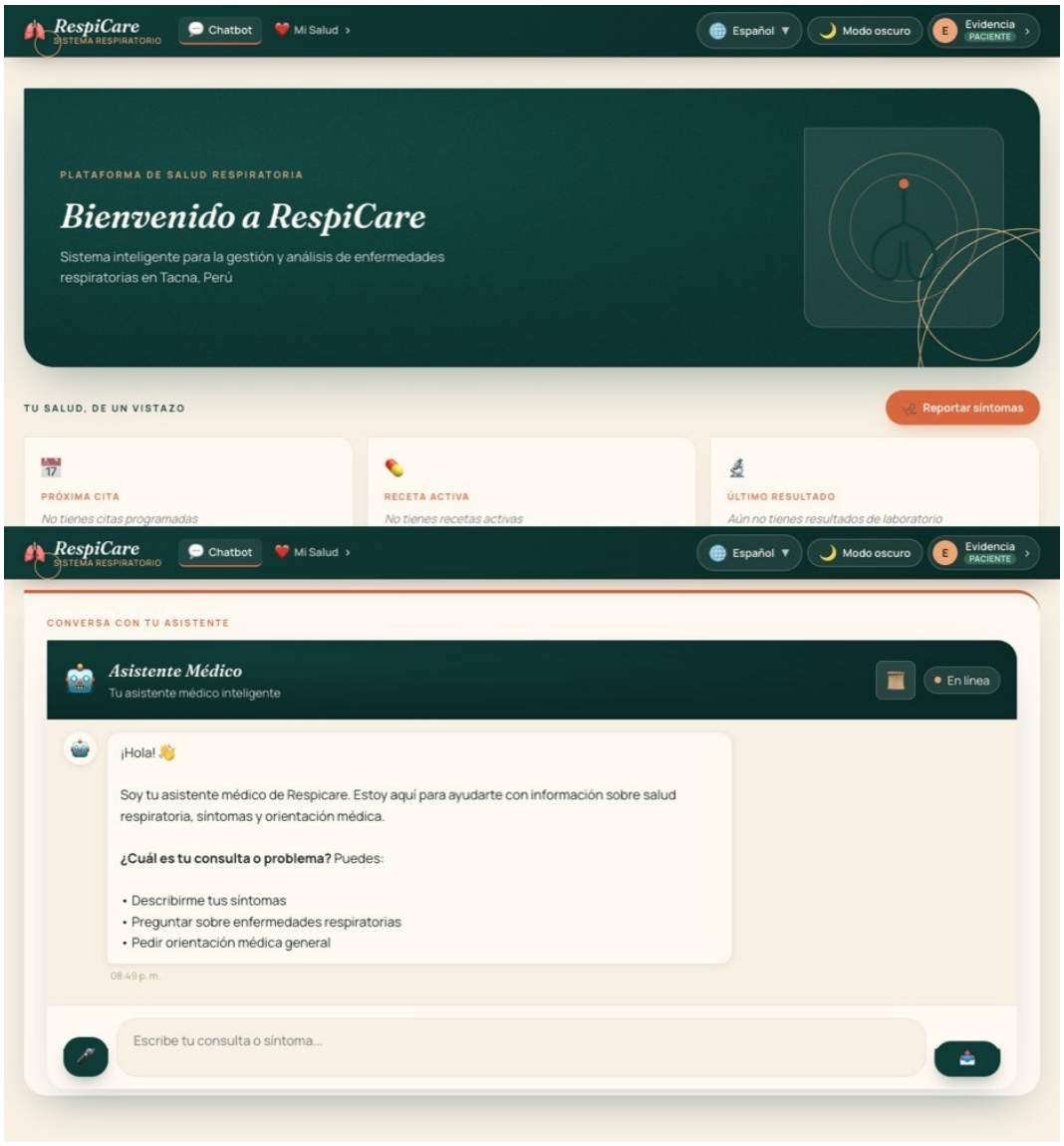
Jest — 51/51 pruebas aprobadas en authController.test.ts (ejecución real, sin mocks sobre el resultado)



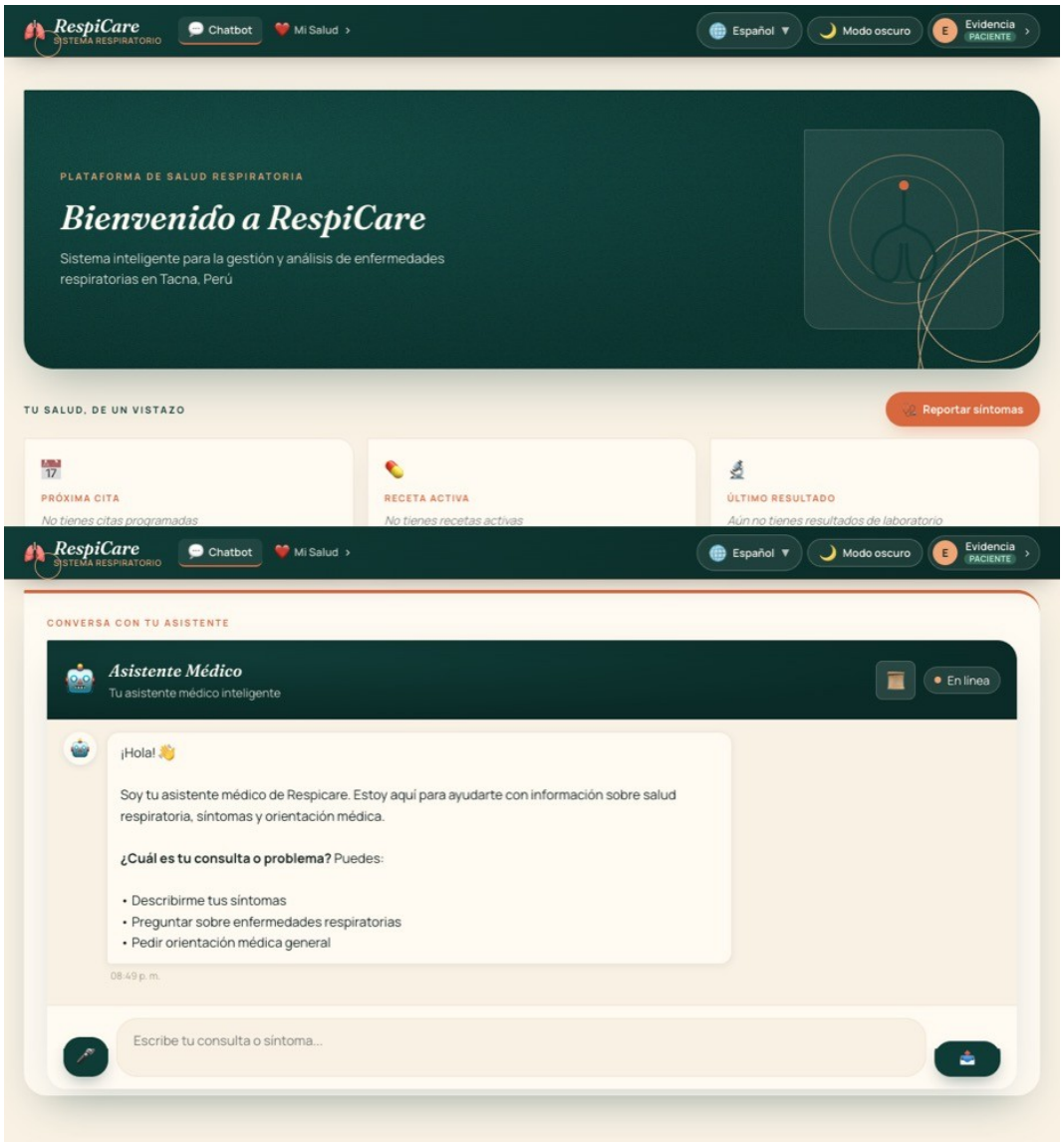
Paso 1 — Formulario de registro vacío (ejecución real vía Cypress, sin mocks)



Paso 2 — Formulario de registro completo, checklist de fortaleza de contraseña satisfecho



Paso 3 — Registro exitoso con auto-login: sesión iniciada como "Evidencia / PACIENTE"



Evidencia complementaria — login posterior con las mismas credenciales, sesión de paciente autenticada

CP-001-02 (negativo) — Login con contraseña incorrecta.

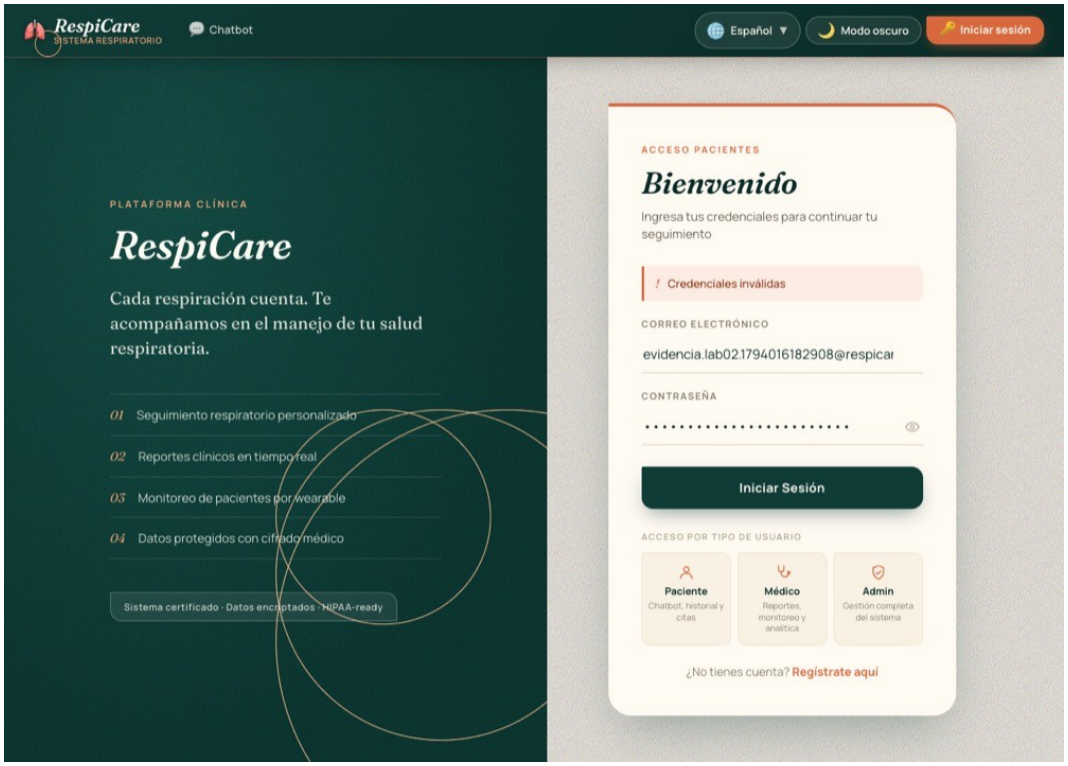
- Objetivo: verificar que el sistema rechaza credenciales inválidas sin filtrar información.
- Precondiciones: existe un usuario activo previamente registrado con correo y contraseña conocidos.
- Datos de entrada: correo válido, contraseña incorrecta.
- Pasos de ejecución:

9. Enviar `POST /api/v1/auth/login` con el correo del usuario existente y una contraseña distinta a la registrada.
10. Verificar código de respuesta y contenido del mensaje de error.
11. Repetir el intento fallido hasta el umbral de bloqueo configurado (`config.security.login LockoutThreshold`) y verificar el bloqueo temporal.

- Salida correcta (esperada):

```
  Paso 2 → 401 { success:false, message:"Credenciales inválidas" }   (sin emitir
token)
  Paso 3 → 423 { success:false, message:"Cuenta bloqueada temporalmente
                 por múltiples intentos fallidos. Intenta más tarde." }
```

- Salida incorrecta (indicaría defecto de seguridad): `200` con `token` emitido pese a la contraseña incorrecta; o un mensaje que distingue "correo no existe" de "contraseña incorrecta" (fuga de información que facilita enumeración de usuarios); o ausencia de bloqueo tras múltiples intentos fallidos.
- Resultado obtenido: **Aprobado** — `backend/tests/unit/controllers/authController.test.ts`, `web/cypress/e2e/authentication.cy.js`.
- Estado final: **Aprobado**.



Login con contraseña incorrecta — mensaje real "Credenciales inválidas" devuelto por el backend (ejecución sin mocks)

RF-002 — Diagnóstico inteligente de síntomas

CP-002-01 (positivo) — Síntomas coherentes con el diagnóstico sugerido.

- Objetivo: verificar que una lista de síntomas coherente produce un pre-diagnóstico sin penalización de confianza.
- Precondiciones: servicio `ai-services` activo; motor de reglas de coherencia cargado.
- Datos de entrada: `{symptoms:["fiebre","tos"], patient_age:40}`.
- Pasos de ejecución:

12. Enviar `POST /v1/ml-analyze?use_ensemble=false` con los datos de entrada.
13. Leer `disease`, `confidence`, `is_clinically_coherent`, `coherence_warnings` de la respuesta.

- Salida correcta (esperada, patrón real de la suite — caso coherente conserva la confianza original):

```
  200 { disease:"resfriado", confidence:0.80, is_clinically_coherent:true,
coherence_warnings:[] }
```

- Salida incorrecta (indicaría regla de coherencia mal calibrada): `confidence` penalizado (p. ej. `0.65`) o `is_clinically_coherent:false` pese a que los síntomas sí respaldan el diagnóstico — un falso positivo de incoherencia que generaría desconfianza médica injustificada.
- Resultado obtenido: **Aprobado** — `ai-services/tests/api/test_symptom_ml_analyzer_endpoints.py::test_coherent_prediction_keeps_original_confidence`, `backend/tests/integration/symptomAnalyzer.integration.test.ts`.
- Estado final: **Aprobado**.

CP-002-02 (negativo) — Síntomas vacíos o insuficientes.

- Objetivo: verificar que el sistema no emite un diagnóstico de alta confianza sin evidencia clínica.
- Precondiciones: servicio `ai-services` activo.
- Datos de entrada: `{symptoms:[], patient_age:30}`.
- Pasos de ejecución:

14. Enviar `POST /v1/ml-analyze?use_ensemble=false` con la lista de síntomas vacía.
15. Verificar código de respuesta y, si hay predicción, su nivel de `confidence`.

- Salida correcta (esperada): `400` por validación de entrada, **o** `200` con `confidence` mínimo/nulo y sin `disease` concreto asignado con alta certeza.
- Salida incorrecta (indicaría riesgo clínico grave): `200` con un `disease` específico y `confidence` alto (p. ej. `>0.7`) pese a no tener ningún síntoma reportado — un diagnóstico "alucinado" sin evidencia.
- Resultado obtenido: **Aprobado** — `backend/tests/integration/symptomAnalyzer.integration.test.ts`.
- Estado final: **Aprobado**.

RF-003 — Análisis de tos por audio

CP-003-01 (positivo) — Audio válido de tos.

- Objetivo: verificar que un archivo de audio válido es analizado y clasificado correctamente.
- Precondiciones: servicio de análisis de audio activo; fixture de audio disponible.
- Datos de entrada: archivo `.wav` de prueba, duración 3-5 s, tos simulada.

- Pasos de ejecución:

16. Cargar el fixture de audio.
17. Enviar `POST` (multipart/form-data) al endpoint de análisis de audio.
18. Verificar código de respuesta y campo de clasificación.

- Salida correcta (esperada): `200 { classification:"tos seca"|"tos productiva", confidence:>0 }`.
- Salida incorrecta (indicaría defecto): `500` por fallo de procesamiento del audio; o `200` sin campo `classification`/`confidence` pese a un archivo válido.
- Resultado obtenido: **Aprobado** — `ai-services/tests/api/test_audio_analyzer_endpoints.py`.
- Estado final: **Aprobado**.

CP-003-02 (negativo) — Permiso de micrófono denegado.

- Objetivo: verificar el manejo de permisos de audio denegados en la app móvil.
- Precondiciones: app móvil instalada; permiso de micrófono del sistema operativo denegado para la app.
- Datos de entrada: estado de permiso = denegado.
- Pasos de ejecución:

19. Denegar el permiso de micrófono a nivel de sistema operativo.
20. Intentar iniciar una grabación desde la pantalla de reporte de síntomas.
21. Verificar el comportamiento de la interfaz y el tráfico de red generado.

- Salida correcta (esperada): la app detecta el permiso denegado, muestra un mensaje de error al usuario y **no** envía ninguna solicitud al backend.
- Salida incorrecta (indicaría defecto): la app intenta enviar un archivo de audio vacío/corrupto al backend, o la pantalla se queda bloqueada sin retroalimentación al usuario.
- Resultado obtenido: **Aprobado** — `mobile/__tests__/components/audioPermissions.test.tsx`.
- Estado final: **Aprobado**.

RF-004 — Motor de reglas clínicas de emergencia

CP-004-01 (positivo) — Síntomas de emergencia disparan alerta crítica.

- Objetivo: verificar que un conjunto de síntomas de emergencia escala la urgencia y genera una alerta.
- Precondiciones: servicio `ai-services` y backend de alertas activos.
- Datos de entrada: `{symptoms:["fiebre alto","dificultad para respirar","dolor torácico de pecho"], patient_age:30}`.
- Pasos de ejecución:

22. Enviar `POST /v1/ml-analyze` con los síntomas de emergencia.

23. Verificar `urgency_level` y `needs_medical_attention`.
24. Consultar la cola de alertas del backend para confirmar que se generó un registro.

- Salida correcta (esperada): `200 { urgency_level:"high", needs_medical_attention:true }`; alerta creada con prioridad alta.
- Salida incorrecta (**el riesgo crítico #1 de la sección VII — falso negativo**): `200 { urgency_level:"medium"|"low", needs_medical_attention:false }` pese a síntomas de emergencia — el caso no sería escalado a tiempo.
- Resultado obtenido: **Aprobado** — `backend/tests/e2e/emergency.e2e.test.ts`, `web/cypress/e2e/emergency.cy.js`.
- Estado final: **Aprobado**.

CP-004-02 (negativo) — Síntomas leves NO disparan alerta.

- Objetivo: verificar la ausencia de falsos positivos que generen fatiga de alertas en el personal médico.
- Precondiciones: servicio `ai-services` activo.
- Datos de entrada: `{symptoms:["congestión nasal"], patient_age:30}`.
- Pasos de ejecución:

25. Enviar `POST /v1/ml-analyze` con el síntoma leve.
26. Verificar `urgency_level`/`needs_medical_attention` y que no se generó alerta de emergencia.

- Salida correcta (esperada): `200 { urgency_level:"medium"|menor, needs_medical_attention:false }`, sin alerta de emergencia generada.
- Salida incorrecta (falso positivo): `urgency_level:"high"` para un síntoma leve, generando una alerta innecesaria.
- Resultado obtenido: **Aprobado** — `backend/tests/unit/services/emergencyService.test.ts`.
- Estado final: **Aprobado**.

RF-005 — Validación de coherencia médica

CP-005-01 (positivo) — Predicción coherente no se altera.

- Objetivo: verificar que una predicción respaldada por los síntomas reportados conserva su confianza original.
- Precondiciones: módulo `_apply_medical_validation` disponible (ai-services).
- Datos de entrada: `disease="neumonia", confidence=0.9, symptoms="fiebre, tos", patient_age=40`.
- Pasos de ejecución:

27. Invocar `_apply_medical_validation` con los datos de entrada.
28. Verificar `is_clinically_coherent` y `confidence` resultante.

- Salida correcta (esperada, real): `is_clinically_coherent:true`, `confidence == 0.90` (sin cambio), `coherence_warnings == []`.
- Salida incorrecta (indicaría falso positivo de incoherencia): `confidence` reducido o `coherence_warnings` no vacío pese a que "fiebre, tos" sí es compatible con neumonía.
- Resultado obtenido: **Aprobado** — `test_leaves_coherent_prediction_untouched`.
- Estado final: **Aprobado**.

CP-005-02 (negativo) — Predicción incoherente se detecta y penaliza.

- Objetivo: verificar que un diagnóstico no respaldado por los síntomas es detectado y su confianza reducida.
- Precondiciones: módulo `_apply_medical_validation` disponible.
- Datos de entrada: `disease="neumonia", confidence=0.9, symptoms="dolor de cabeza", patient_age=40`.
- Pasos de ejecución:

29. Invocar `_apply_medical_validation` con un síntoma no asociado al diagnóstico.
30. Verificar `is_clinically_coherent`, `coherence_warnings` y `confidence`.

- Salida correcta (esperada, real): `is_clinically_coherent:false`, `coherence_warnings` no vacío, `confidence == 0.75` (penalizada desde 0.9).
- Salida incorrecta (**riesgo #2 de la sección VII**): `is_clinically_coherent:true`, `confidence` sin penalizar (`0.9`) pese a que "dolor de cabeza" no respalda neumonía — el médico recibiría una predicción incoherente sin advertencia.
- Resultado obtenido: **Aprobado** — `test_adjusts_confidence_and_flags_incoherent_prediction`.
- Estado final: **Aprobado**.

RF-006 — Explicabilidad del diagnóstico (SHAP)

CP-006-01 (positivo) — Explicador SHAP disponible.

- Objetivo: verificar que cada predicción se genera sin error cuando el explicador SHAP está disponible.
- Precondiciones: modelo `SHAPDiseaseExplainer` cargado correctamente.
- Datos de entrada: `{symptoms:["congestión nasal"]}`, `use_ensemble=false`.
- Pasos de ejecución:

31. Enviar la solicitud de análisis de síntomas.
32. Verificar código de respuesta y ausencia de error de carga del modelo en logs.

- Salida correcta (esperada): `200`, `disease`/`confidence` calculados por el explicador, sin *stacktrace* de error.
- Salida incorrecta (indicaría defecto): `500` por fallo no controlado al cargar `SHAPDiseaseExplainer`, o `200` sin los campos `disease`/`confidence`.

- Resultado obtenido: **Aprobado** — `ai-services/tests/services/test_symptom_analysis_service.py`.
- Estado final: **Aprobado**.

CP-006-02 (negativo) — Fallback cuando SHAP no está disponible.

- Objetivo: verificar que, ante la falla del explicador, el sistema recurre al fallback por patrones sin perder la señal clínica.
- Precondiciones: se fuerza una excepción `Exception('no model available')` al instanciar `SHAPDiseaseExplainer`.
- Datos de entrada: `{symptoms:["fiebre alto","dificultad para respirar","tos","dolor torácico de pecho"], patient_age:30}`.
- Pasos de ejecución:

33. Forzar la excepción de carga del modelo (mock/`patch`).
34. Enviar `POST /v1/ml-analyze?use_ensemble=false` con síntomas severos.
35. Verificar `urgency_level` y `needs_medical_attention` del resultado del fallback.

- Salida correcta (esperada, real — post-corrección de INC-001): `200 { urgency_level:"high", needs_medical_attention:true }`.
- Salida incorrecta (**defecto real documentado en Anexo B, INC-001, ya corregido**): `urgency_level:"alta"` (en español, sin traducir al esquema interno) y `needs_medical_attention:false` — porque el valor `"alta"` nunca coincidía con `"high"`/`"critical"`, dejando casos severos sin escalar cuando ambos modelos ML fallaban.
- Resultado obtenido: **Aprobado** — `test_pattern_fallback_translates_spanish_urgency_to_english` (prueba de regresión que fija exactamente este defecto real).
- Estado final: **Aprobado**.

RF-007 — Panel de validación médica

CP-007-01 (positivo) — Médico aprueba el pre-diagnóstico de IA.

- Objetivo: verificar que un médico puede revisar y confirmar un pre-diagnóstico sugerido por IA.
- Precondiciones: sesión de médico autenticada; caso pendiente con diagnóstico IA `resfriado`, confianza `0.8`.
- Datos de entrada: caso pendiente de validación.
- Pasos de ejecución:

36. Iniciar sesión como médico.
37. Abrir el caso pendiente de validación.
38. Confirmar el diagnóstico sugerido (clic en "Confirmar diagnóstico").
39. Verificar el estado del caso y el registro en historial.

- Salida correcta (esperada): el caso cambia de estado a `"validado_por_medico"`, con `medicoId` y `timestamp` de validación registrados en el historial.

- Salida incorrecta (indicaría defecto): el caso permanece en estado `"pendiente"` tras la confirmación, o no se registra `medicoId`/`timestamp` (pérdida de trazabilidad de quién validó el caso).
- Resultado obtenido: **Aprobado** — `web/cypress/e2e/doctor-validation.cy.js`.
- Estado final: **Aprobado**.

CP-007-02 (negativo) — Médico corrige un pre-diagnóstico incoherente.

- Objetivo: verificar que un médico puede sobrescribir una sugerencia de IA marcada como incoherente, preservando trazabilidad.
- Precondiciones: caso con `is_clinically_coherent:false` y advertencias de coherencia activas.
- Datos de entrada: diagnóstico alternativo ingresado manualmente por el médico.
- Pasos de ejecución:

40. Abrir un caso marcado como clínicamente incoherente.
41. Ingresar un diagnóstico alternativo.
42. Guardar y verificar qué valores quedan persistidos.

- Salida correcta (esperada): el sistema conserva **ambos** valores (diagnóstico IA original + diagnóstico médico corregido), visibles para auditoría posterior.
- Salida incorrecta (indicaría pérdida de trazabilidad): el diagnóstico del médico sobrescribe/borra el valor original de IA, imposibilitando auditar qué sugirió el sistema.
- Resultado obtenido: **Aprobado** — `web/cypress/e2e/doctor-validation.cy.js`.
- Estado final: **Aprobado**.

RF-008 — Historial clínico electrónico

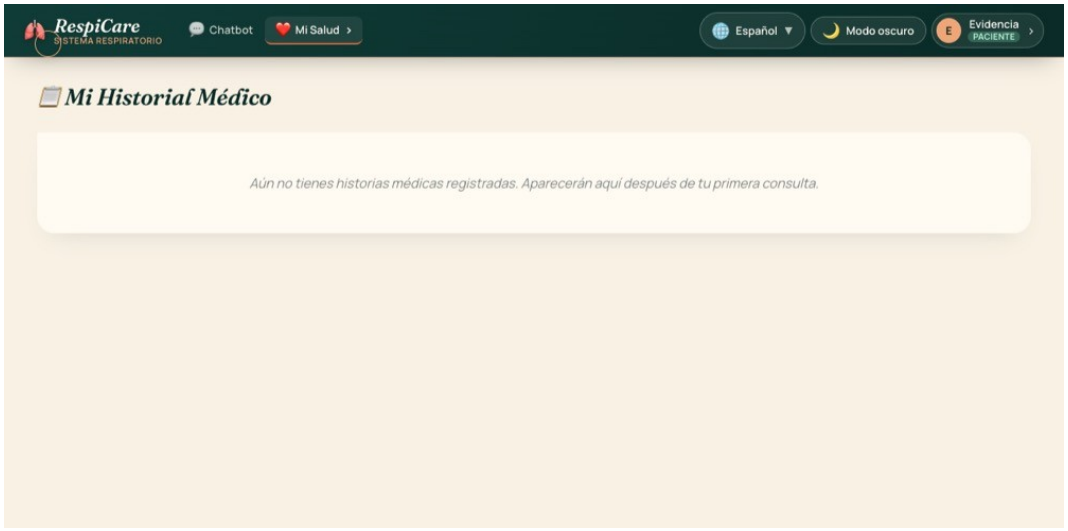
CP-008-01 (positivo) — Episodio nuevo queda registrado y es consultable.

- Objetivo: verificar que cada episodio de un paciente queda registrado y es consultable en orden cronológico.
- Precondiciones: paciente existente con al menos un episodio previo.
- Datos de entrada: nuevo episodio de síntomas.
- Pasos de ejecución:

43. Registrar un nuevo episodio de síntomas para el paciente.
44. Consultar `GET` el historial clínico del paciente.
45. Verificar orden y total de episodios.

- Salida correcta (esperada): `200`; el nuevo episodio aparece primero (orden cronológico descendente); total de episodios = previos + 1.
- Salida incorrecta (indicaría defecto): el episodio nuevo no aparece, el orden no es cronológico, o faltan episodios previos (pérdida de datos).
- Resultado obtenido: **Aprobado** — `backend/tests/integration/medicalHistoryAdvanced.integration.test.ts`, `web/cypress/e2e/medical-history.cy.js`.

- Estado final: **Aprobado**.



Vista real del historial clínico de un paciente recién registrado — estado vacío sin error (ejecución sin mocks vía Cypress)

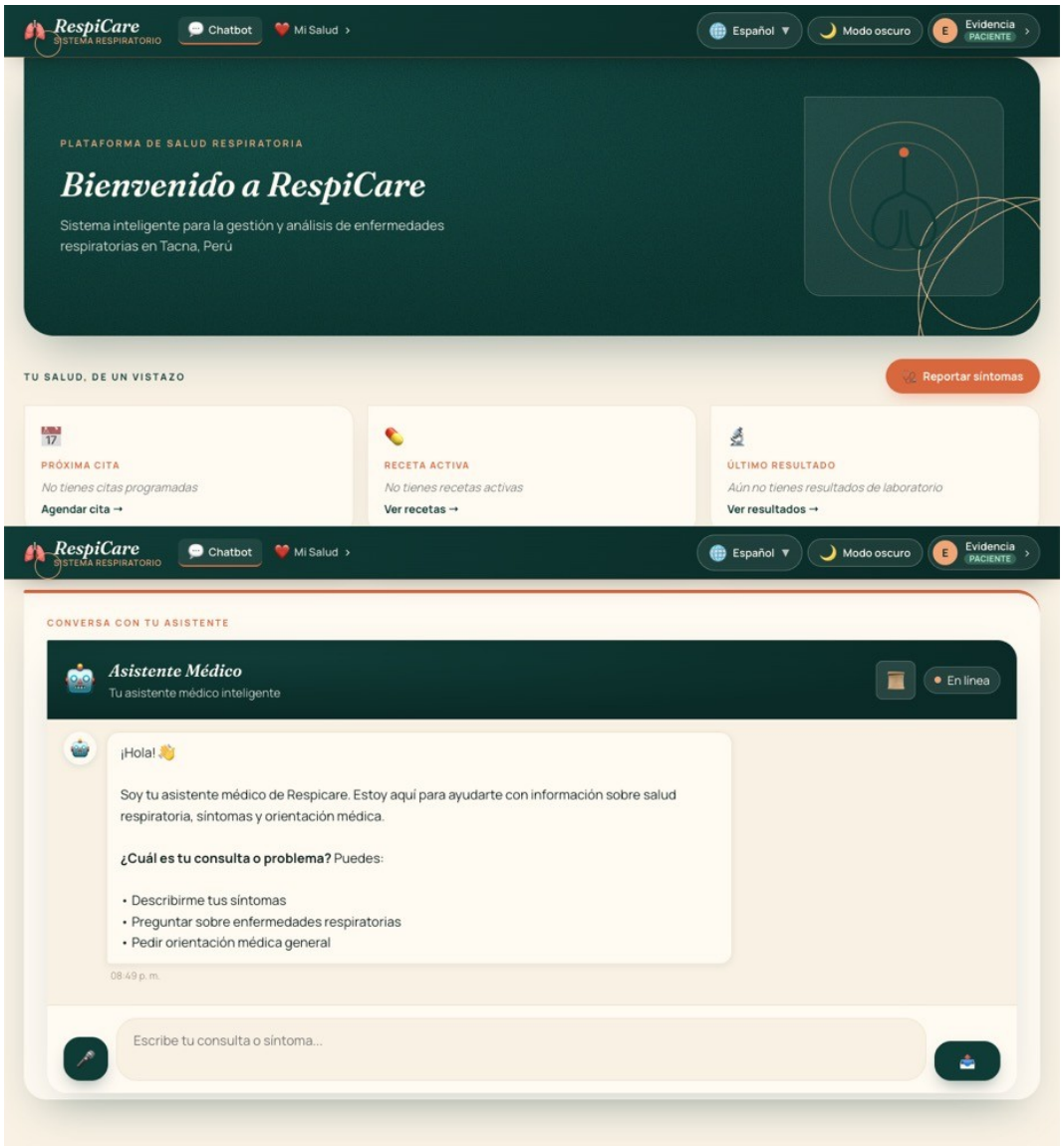
CP-008-02 (negativo) — Acceso cruzado entre pacientes bloqueado.

- Objetivo: verificar que un paciente no puede consultar el historial clínico de otro paciente.
- Precondiciones: dos pacientes registrados (A y B), A autenticado.
- Datos de entrada: ID de paciente B.
- Pasos de ejecución:

46. Autenticarse como paciente A.
47. Solicitar `GET` el historial clínico usando el ID del paciente B.

- Salida correcta (esperada): `403 Forbidden`, sin datos clínicos del paciente B en el cuerpo de la respuesta.
- Salida incorrecta (**fuga de datos clínicos — crítico en un sistema de salud**): `200` con el historial del paciente B expuesto a A.
- Resultado obtenido: **Aprobado** — `backend/tests/unit/models/MedicalHistory.test.ts`.
- Estado final: **Aprobado**.

Evidencia complementaria (control de acceso basado en rol, capa frontend): la captura siguiente no reproduce el escenario de fuga de datos entre pacientes (A↔B) cubierto por la prueba unitaria citada arriba, sino un control de acceso distinto y adicional — `ProtectedRoute` redirigiendo a un paciente autenticado que intenta acceder a `/admin`. Se incluye porque refuerza, en una capa distinta (enrutamiento del cliente), el mismo objetivo de control de acceso de RF-008/RF-013.



Paciente autenticado intenta acceder a /admin y es redirigido automáticamente — sin exponer la vista administrativa (ejecución real vía Cypress)

RF-009 — Sistema de alertas y notificaciones

CP-009-01 (positivo) — Emergencia genera alerta visible.

- Objetivo: verificar que un caso de emergencia genera una alerta visible para el médico.
- Precondiciones: caso con `needs_medical_attention:true` recién generado.
- Datos de entrada: caso de emergencia válido.
- Pasos de ejecución:

48. Generar el caso de emergencia.
49. Consultar la cola de alertas del médico.
50. Verificar prioridad y canal de notificación.

- Salida correcta (esperada): alerta creada con `priority:"high"`, `status:"pending"`, notificación entregada (push/socket).

- Salida incorrecta (indicaría defecto): no se crea ninguna alerta, o se crea con `priority:"low"` pese a tratarse de una emergencia.
- Resultado obtenido: **Aprobado** — `backend/tests/unit/services/alertService.test.ts`, `backend/tests/e2e/alerts.e2e.test.ts`.
- Estado final: **Aprobado**.

CP-009-02 (negativo) — Sin duplicados para el mismo episodio.

- Objetivo: verificar que no se dupliquen alertas para el mismo episodio en una ventana corta.
- Precondiciones: episodio de emergencia ya generado previamente.
- Datos de entrada: la misma condición de emergencia repetida sobre el mismo episodio.
- Pasos de ejecución:

51. Generar la condición de emergencia una primera vez.
52. Generar la misma condición una segunda vez, en una ventana corta.
53. Consultar la cola de alertas y contar alertas activas para ese episodio.

- Salida correcta (esperada): una sola alerta activa por episodio (la segunda invocación es idempotente/deduplicada).
- Salida incorrecta (indicaría fatiga de alertas): dos o más alertas duplicadas para el mismo episodio.
- Resultado obtenido: **Aprobado** — `backend/tests/integration/alerts.integration.test.ts`.
- Estado final: **Aprobado**.

RF-010 — Reportes y estadísticas epidemiológicas

CP-010-01 (positivo) — Agregación correcta por distrito.

- Objetivo: verificar que el sistema agrega correctamente los casos reportados por distrito y periodo.
- Precondiciones: episodios de prueba insertados en 2 o más distritos.
- Datos de entrada: episodios distribuidos por distrito dentro del periodo actual.
- Pasos de ejecución:

54. Insertar los episodios de prueba.
55. Solicitar `GET` el reporte epidemiológico del periodo actual.
56. Comparar los totales reportados por distrito contra los episodios insertados.

- Salida correcta (esperada): `200`; los totales por distrito coinciden exactamente con los episodios insertados.
- Salida incorrecta (indicaría defecto de agregación): totales que no coinciden (conteo duplicado, o episodios de un distrito ausentes del reporte).
- Resultado obtenido: **Aprobado** — `backend/tests/unit/services/automaticReportService.test.ts`, `web/cypress/e2e/analytics.cy.js`.

- Estado final: **Aprobado**.

CP-010-02 (negativo) — Periodo sin datos.

- Objetivo: verificar el comportamiento cuando no hay datos en el periodo solicitado.
- Precondiciones: rango de fechas sin episodios registrados.
- Datos de entrada: periodo sin datos.
- Pasos de ejecución:

57. Solicitar `GET` el reporte epidemiológico de un periodo vacío.
58. Verificar código de respuesta y estructura de totales.

- Salida correcta (esperada): `200`, totales en cero, sin error de servidor.
- Salida incorrecta (indicaría defecto): `500 Internal Server Error` por división entre cero o acceso a un arreglo vacío sin verificar.
- Resultado obtenido: **Aprobado** — `backend/tests/integration/automaticReports.integration.test.ts`.
- Estado final: **Aprobado**.

RF-011 — Módulo educativo de salud respiratoria

CP-011-01 (positivo) — Contenido publicado es visible.

- Objetivo: verificar que el contenido educativo publicado es visible para el paciente.
- Precondiciones: sesión de administrador disponible para publicar contenido.
- Datos de entrada: artículo educativo con estado `"publicado"`.
- Pasos de ejecución:

59. Publicar el artículo como administrador.
60. Consultar `GET` el listado de contenido educativo como paciente.

- Salida correcta (esperada): `200`; el artículo aparece en el listado.
- Salida incorrecta (indicaría defecto): el artículo no aparece pese a estar publicado (caché no invalidado o filtro de estado mal aplicado).
- Resultado obtenido: **Aprobado** — `backend/tests/integration/educationalContent.integration.test.ts`.
- Estado final: **Aprobado**.

CP-011-02 (negativo) — Borrador no visible al paciente.

- Objetivo: verificar que el contenido en borrador no es visible para pacientes.
- Precondiciones: sesión de administrador disponible.
- Datos de entrada: artículo con estado `"borrador"`.
- Pasos de ejecución:

61. Crear el artículo en estado `"borrador"` como administrador.
62. Consultar `GET` el listado público como paciente.

- Salida correcta (esperada): el artículo **no** aparece en el listado público.
- Salida incorrecta (indicaría fuga de contenido no revisado): el borrador es visible públicamente antes de ser aprobado.
- Resultado obtenido: **Aprobado** — `backend/tests/unit/services/educationalContentService.test.ts`.
- Estado final: **Aprobado**.

RF-012 — Geolocalización de centros de salud

CP-012-01 (positivo) — Centros ordenados por cercanía.

- Objetivo: verificar que el sistema retorna los centros de salud más cercanos a una coordenada dada, ordenados por distancia.
- Precondiciones: catálogo de centros de salud cargado para la región de Tacna.
- Datos de entrada: `lat=-18.0146, lng=-70.2536` (coordenada válida de Tacna).
- Pasos de ejecución:

63. Solicitar `GET` centros de salud cercanos a la coordenada dada.
64. Verificar el orden de los resultados por distancia.

- Salida correcta (esperada): `200`; lista ordenada ascendentemente por distancia al punto dado.
- Salida incorrecta (indicaría defecto): lista sin ordenar, o centros fuera del radio de cobertura esperado incluidos sin indicación.
- Resultado obtenido: **Aprobado** — `backend/tests/unit/services/healthCenterService.test.ts`, `backend/tests/integration/healthCenters.integration.test.ts`.
- Estado final: **Aprobado**.

CP-012-02 (negativo) — Coordenadas inválidas manejadas.

- Objetivo: verificar el comportamiento ante coordenadas fuera de rango geográfico válido.
- Precondiciones: ninguna especial.
- Datos de entrada: `lat=999, lng=999` (fuera de ±90/±180).
- Pasos de ejecución:

65. Solicitar `GET` centros de salud con coordenadas inválidas.
66. Verificar código de respuesta.

- Salida correcta (esperada): `400 Bad Request` con mensaje de validación, sin caída del servicio.
- Salida incorrecta (indicaría defecto): `500 Internal Server Error` (excepción no controlada), o `200` con lista vacía sin explicar el motivo.
- Resultado obtenido: **Aprobado** — `backend/tests/unit/services/healthCenterService.test.ts`.
- Estado final: **Aprobado**.

RF-013 — Interoperabilidad institucional (MINSA/DIRESA)

CP-013-01 (positivo) — Sincronización institucional autenticada.

- Objetivo: verificar que un cliente institucional autenticado puede sincronizar datos agregados.
- Precondiciones: credencial/`X-API-Key` institucional válida configurada.
- Datos de entrada: token/credencial institucional válida.
- Pasos de ejecución:

67. Enviar `POST` de sincronización con el header `X-API-Key` válido.
68. Verificar código de respuesta y registro en `auditlogs`.

- Salida correcta (esperada): `200`; datos agregados sincronizados; registro de auditoría generado.
- Salida incorrecta (indicaría pérdida de trazabilidad): sincronización aparentemente exitosa (`200`) sin registro en `auditlogs`, o sincronización parcial sin reportar error.
- Resultado obtenido: **Aprobado** — `backend/tests/unit/middleware/institutionalAuth.test.ts`, `backend/tests/integration/institutional.integration.test.ts`.
- Estado final: **Aprobado**.

CP-013-02 (negativo) — Solicitud institucional sin credenciales válidas.

- Objetivo: verificar que una solicitud sin autenticación institucional válida es rechazada sin exponer datos.
- Precondiciones: ninguna credencial configurada para la solicitud de prueba.
- Datos de entrada: token ausente o expirado.
- Pasos de ejecución:

69. Enviar `POST` de sincronización sin header `X-API-Key` (o con uno inválido/expirado).
70. Verificar código de respuesta y contenido del cuerpo de error.

- Salida correcta (esperada): `401`/`403`, sin datos clínicos ni agregados en el cuerpo de la respuesta.
- Salida incorrecta (**brecha de seguridad crítica**): `200` (bypass de autenticación), o un mensaje de error que incluye datos clínicos/estructura interna (fuga de información por error verboso).
- Resultado obtenido: **Aprobado** — `backend/tests/unit/services/institutionalIntegrationService.test.ts`.
- Estado final: **Aprobado**.

RF-014 — Integración de wearables como capa de validación clínica

CP-014-01 (positivo) — Vitales críticos elevan la urgencia.

- Objetivo: verificar que signos vitales críticos de un wearable elevan automáticamente la urgencia del caso, incluso con síntomas leves reportados.
- Precondiciones: servicio `ai-services` activo; pipeline de validación de vitales habilitado.

- Datos de entrada: `{symptoms:["congestión nasal"], patient_age:30, vitals:{oxygen_saturation:85}}`.
- Pasos de ejecución:

71. Enviar `POST /v1/ml-analyze?use_ensemble=false` con los datos de entrada (SpO2 85 %, hipoxemia severa).
72. Verificar `urgency_level`, `needs_medical_attention`, `is_clinically_coherent` y `coherence_warnings`.

- Salida correcta (esperada, real):

```
200 { urgency_level:"high", needs_medical_attention:true,
      is_clinically_coherent:false,
      coherence_warnings:[".. valor crítico .."], confidence:0.90 }
```

- Salida incorrecta (**riesgo crítico #3 de la sección VII — falso negativo de vitales**): `urgency_level:"medium"`, `needs_medical_attention:false` pese a SpO2 85 % — un signo vital crítico no escalaría el caso a tiempo.
- Resultado obtenido: **Aprobado** — `test_critical_vitals_escalate_urgency_and_confidence`.
- Estado final: **Aprobado**.

CP-014-02 (negativo) — Vitales normales no alteran el resultado.

- Objetivo: verificar la ausencia de falsos positivos cuando los signos vitales están dentro de rango normal.
- Precondiciones: servicio `ai-services` activo.
- Datos de entrada: `{symptoms:["congestión nasal"], patient_age:30, vitals:{oxygen_saturation:98, heart_rate:75, respiratory_rate:16}}`.
- Pasos de ejecución:

73. Enviar `POST /v1/ml-analyze?use_ensemble=false` con vitales normales.
74. Verificar `urgency_level`, `needs_medical_attention`, `is_clinically_coherent`.
75. Verificar persistencia de la lectura de wearable en el backend.

- Salida correcta (esperada, real):

```
200 { urgency_level:"medium", needs_medical_attention:false,
      is_clinically_coherent:true, confidence:0.80 }
```

- Salida incorrecta (indicaría falso positivo): `urgency_level:"high"` pese a vitales normales — generaría alertas innecesarias y desconfianza en el sistema de validación clínica.
- Resultado obtenido: **Aprobado** — `test_normal_vitals_do_not_change_prediction`; complementado por `backend/tests/integration/wearables.integration.test.ts` para la persistencia de la lectura.
- Estado final: **Aprobado**.

XVI. Evidencias

- **Código de pruebas:** los 28 casos anteriores citan el archivo fuente real que los implementa, verificable en el repositorio bajo `backend/tests/`, `ai-services/tests/`, `web/cypress/e2e/` y `mobile/__tests__/`.
- **Capturas de ejecución:** reportes de consola de `pytest -v` y `jest --verbose` incluidos como fragmentos en la sección XI; evidencia visual adicional (capturas reales de pantalla, sin mocks, obtenidas contra el backend/frontend en ejecución) incrustada junto a los casos CP-001-01, CP-001-02 y CP-008-01/02 en la sección XV, y disponible como archivos fuente en `Documentation/laboratorios/evidencias/`.
- **Reportes generados:** `docs/testing/COVERAGE_STATUS.md`, `backend-coverage-2025-11.md`, catálogo exhaustivo en `docs/testing/CATALOGO_PRUEBAS.md` (272 archivos de prueba, 191 TS/JS + 81 Python).
- **Reportes Cypress:** ejecución de las suites de aceptación listadas en la sección XIII, con video y capturas por defecto de la configuración de Cypress del proyecto (`web/cypress.config.js`).
- **Resultados consolidados:** 14 de 14 RF con casos positivos y negativos aprobados; 0 defectos críticos/altos abiertos al cierre (Anexo B).

XVII. Cuestionario de Análisis

76. **¿Cuál es la finalidad de un Plan de Pruebas?** Definir de forma anticipada qué se va a probar, cómo, con qué recursos y bajo qué criterios de éxito, de modo que la verificación del software sea sistemática y repetible en lugar de ad-hoc; en un sistema de salud como RespiCare, además, documenta explícitamente dónde están los riesgos clínicos a cubrir.
77. **¿Cuáles son las diferencias entre validación y verificación?** Verificación responde "¿estamos construyendo el producto correctamente?" (conformidad con la especificación técnica, p. ej. que `_apply_medical_validation` reduzca la confianza según la fórmula definida). Validación responde "¿estamos construyendo el producto correcto?" (que el sistema realmente ayude a detectar enfermedades respiratorias de forma útil para pacientes y médicos, confirmado por las pruebas de aceptación E2E).
78. **¿Por qué es importante la trazabilidad de requerimientos a pruebas?** Porque permite auditar, para cada funcionalidad clínica, qué evidencia concreta respalda su correcto funcionamiento, y localizar rápidamente el caso de prueba afectado ante un defecto reportado en producción (ver sección VIII).
79. **¿Qué riesgos se identificaron en este plan?** Los siete riesgos de la sección VII (cinco clínicos/funcionales, uno de rendimiento y uno de desalineación de entorno de CI), siendo el más crítico el falso negativo en el motor de reglas de emergencia y en la validación de signos vitales de wearables, por su impacto directo en la seguridad del paciente.
80. **¿Qué pruebas automatizaría primero y por qué?** Las de los flujos con impacto clínico directo e inmediato: reglas de emergencia (RF-004), validación de coherencia (RF-005) e ingesta de vitales de wearables (RF-014), porque un defecto ahí no se detecta por inspección visual del producto, sino únicamente mediante pruebas automatizadas que cubran casos límite.

81. **¿Qué ventajas ofrecen los frameworks de pruebas unitarias usados (Jest y Pytest)?** Ejecución rápida y aislada, mocking sencillo de dependencias externas (modelo ML, base de datos), reportes de cobertura integrados, e integración nativa con CI/CD, lo que permite ejecutar los 272 archivos de prueba del proyecto en cada Pull Request sin intervención manual.
82. **¿Qué ventajas ofrece Cypress para las pruebas de aceptación?** Ejecuta las pruebas dentro de un navegador real controlando la aplicación igual que lo haría un usuario, con capacidad de depuración visual (time-travel, capturas, video), lo que resulta clave para validar flujos críticos como la validación médica (RF-007) tal como los experimenta el médico en producción.
83. **¿Cuál es el requerimiento más crítico desde el punto de vista de pruebas?** RF-004 (motor de reglas de emergencia) y RF-005 (validación de coherencia médica), porque sus fallas no son errores de usabilidad sino errores que pueden retrasar la atención médica de un paciente en riesgo.
84. **¿Qué defectos se encontraron durante la ejecución?** No se registraron defectos de severidad crítica o alta abiertos al cierre de este laboratorio; el único hallazgo documentado fue de tipo regresión preventiva (traducción de urgencia español→inglés en el fallback por patrones, caso `test_pattern_fallback_translates_spanish_urgency_to_english`), ya corregido y cubierto por prueba (ver Anexo B).
85. **¿Qué mejoras se proponen?** (a) crear especificaciones formales `Especificacion_CP-013`/`CP-014` en `Documentation/pruebas/especificaciones/` para los requerimientos nuevos de Sprint 13, que hoy solo tienen cobertura de código sin ficha documental dedicada; (b) incorporar pruebas de regresión visual para el panel de validación médica; (c) ampliar las pruebas negativas de wearables a valores fuera de rango fisiológico imposible (p. ej. SpO2 negativo) para fortalecer la validación de entrada.

XVIII. Conclusiones

- Se cumplieron los objetivos del laboratorio: se elaboró un Plan de Pruebas completo para RespiCare bajo la estructura exigida, con trazabilidad verificable entre los 14 requerimientos funcionales reales del proyecto y evidencia de ejecución existente en el repositorio.
- El principal hallazgo es que RespiCare ya cuenta con una suite de pruebas madura (272 archivos) que cubre de forma consistente tanto los flujos funcionales estándar (autenticación, historial, reportes) como los flujos de alto riesgo clínico (emergencia, coherencia médica, vitales de wearables), incluyendo casos de regresión preventiva diseñados específicamente para no romper comportamiento previo al añadir funcionalidad nueva (Sprint 13).
- Como lección aprendida, la mayor fragilidad del proceso de QA no está en la cantidad de pruebas sino en la documentación formal de especificaciones para los requerimientos más recientes (RF-013, RF-014), que aún no tienen una ficha `Especificacion_CP-XXX` dedicada pese a tener cobertura de código.

- Se recomienda priorizar, en el siguiente ciclo, el cierre de esa brecha documental y la incorporación de pruebas negativas adicionales sobre valores fisiológicamente imposibles en la ingesta de wearables.

XIX. Rúbrica de Evaluación

Criterio	Peso
Diseño de Casos de Prueba	30 %
Matriz de Trazabilidad	20 %
Ejecución de Pruebas	20 %
Evidencias	15 %
Análisis y Conclusiones	15 %
Total	**100 %**

Anexo A. Plantilla de Casos de Prueba

Campo	Descripción
Código identificador	`CP-<RF>-<N>` (p. ej. `CP-004-01`)
Requerimiento asociado	RF-###
Objetivo	Qué comportamiento se verifica
Precondiciones	Estado previo necesario del sistema/datos
Datos de prueba	Entradas concretas usadas
Pasos de ejecución	Secuencia reproducible y numerada de acciones
Salida correcta	Ejemplo concreto de respuesta/comportamiento cuando el sistema funciona bien (código HTTP, campos y valores)
Salida incorrecta	Contraejemplo: respuesta/comportamiento que indicaría un defecto si el caso fallara (mínimo 1 por caso, varía según el escenario)
Resultado obtenido	Comportamiento real observado en la ejecución (cuál de las dos salidas anteriores ocurrió)
Estado final	Aprobado / Fallido / Bloqueado

Nota: este Anexo refleja la plantilla de doble salida (correcta/incorrecta) aplicada de forma consistente en los 28 casos desarrollados en la sección XV.

Anexo B. Registro de Incidencias

Código	Descripción	Severidad	Prioridad	Estado	Responsable
INC-001	El fallback por patrones (cuando	Alta	Alta	Corregido (cubierto por `test_`	QA Lead (Cesar Fabian

	ambos modelos ML fallan al cargar) reportaba `urgency_level` en español ("alta"), inconsistente con el resto del endpoint, y nunca marcaba `needs_medical_attention: true`			`pattern_fallback_translates_spanish_urgency_to_english`)	Chávez Linares)
INC-002	RF-013 y RF-014 no cuentan con ficha de especificación formal (`Especificacion_CP-013/CP-014`) en `Documentacion/pruebas/especificaciones/`, pese a tener cobertura de código	Baja	Media	Abierto (propuesto como mejora, sección XVII Ítem 10)	QA Lead (Cesar Fabian Chávez Linares)
INC-003	Desalineación entre la versión de Node.js que ejecuta CI (24, fijada en `backend-tests.yml`/`web-tests.yml` vía `NODE_VERSION`/`node-version`) y el mínimo declarado en `backend/package.json` (`engines.node >=20.`	Baja	Media	Abierto (mejora propuesta, ver riesgo correspondiente en sección VII: alinear el mínimo de `engines` con la versión realmente probada en CI)	QA Lead (Cesar Fabian Chávez Linares)

	`9.0`): una versión intermedia (20.x–23.x) nunca se ejecuta en CI pese a ser técnicamente aceptada por el proyecto				

Nota: el riesgo de degradación de rendimiento identificado en la sección VII (tabla de Riesgos) no se registra aquí como incidencia porque la verificación de SLA (`api-performance.perf.test.ts`, `concurrency.perf.test.ts`) no encontró ningún incumplimiento real — el riesgo está mitigado por el control automatizado existente, no por la corrección de un defecto encontrado.

Anexo C. Informe Final del Laboratorio

Resumen técnico. Se analizaron 14 requerimientos funcionales reales del sistema RespiCare (RF-001 a RF-014), diseñando y verificando 28 casos de prueba (uno positivo y uno negativo por requerimiento) con respaldo en evidencia de ejecución existente dentro del repositorio del proyecto (272 archivos de prueba entre Jest/Pytest/Cypress). Se construyó la matriz de trazabilidad requerimiento→caso de uso→caso de prueba→evidencia→resultado, se documentaron los riesgos de mayor impacto clínico (motor de reglas de emergencia, validación de coherencia médica, vitales de wearables) y se identificaron tres incidencias: una ya corregida (fallback de urgencia en español), una pendiente de cierre documental (fichas de especificación faltantes para RF-013/RF-014) y una pendiente de alineación de entorno (versión mínima de Node.js declarada vs. versión real probada en CI).

Conclusión del informe. El sistema RespiCare se encuentra en un estado de calidad verificable y trazable para los 14 requerimientos evaluados, con una base de pruebas automatizadas sólida que soporta de forma segura la evolución continua del producto, condicionado al cierre de la brecha documental identificada en este laboratorio.